

THESYS+ — Explained So a Child Can Understand It

This document walks through the THESYS+ codebase one file at a time, in plain language. Think of the whole app as a **restaurant**:

- The **frontend** (in `frontend/`) is the dining room — what you see and touch.
- The **backend** (in `backend/`) is the kitchen — where the real work happens.
- The **code** is the set of written instructions the restaurant runs on.

Each file below is explained line by line. The goal is understanding, not jargon.

How to read the explanations

- A file is introduced with `## path/to/file`.
 - Inside, we go through the file a few lines at a time, like a story.
 - Code words are shown in backticks.
 - Big ideas get a little emoji so they are easy to spot.
-

Part 1 — Frontend foundation

frontend/src/main.jsx — "Turn on the lights" ■

This is where the website starts when you open it.

- **Line 1:** `import { createRoot } from 'react-dom/client';` — Get the tool that plants a React app onto a real web page.
- **Line 2:** `import './index.css';` — Also bring the stylesheet — the book of colors, fonts, and spacing rules.
- **Lines 3–6:** `import App ... ThemeProvider ... AuthProvider ... ToastProvider` — Bring in the main screen (App) and three helper "wrappers" that remember things: Theme (light/dark mode), Auth (are you logged in?), Toast (little pop-up messages).
- **Lines 8–16 (the NOTE):** A warning to future programmers: *We turned OFF React's "StrictMode" on purpose.* Why? In testing mode React runs some code twice. That double-run made the login system think a bad guy was replaying a stolen key, so it kicked everyone out on every page load. So they removed it. (A real bug they fixed!)
- **Lines 17–25:** Take the spot on the page called `root` and draw this ****nest of boxes**** inside it: Theme on the outside, then Auth, then Toast, and right in the middle, the App (all the actual pages). Think of it like: Winter coat (Theme) → sweater (Auth) → shirt (Toast) → you (App) in the middle. Everything inside gets to use what the outside provides.

frontend/src/App.jsx — "The map of the building" ■■

This file decides **which page shows when you visit a web address**.

- **Lines 1–14 (comment):** Explains the plan: the home page / stands alone, all the

sign-in-style pages share one frame, and a single "bouncer" guards the private pages.

- **Lines 16–37:** A big list of `import` lines — "Bring every page and every helper we might need" (`LandingPage`, `SignInPage`, `RepositoryPage`, etc.).
- **Lines 42–65** `NotFound`: If someone types a garbage web address, show a friendly 404 page with the logo and a "Back to Home" button. Like a "Wrong door!" sign.
- **Lines 70–89** `ProtectedRoute`: A bouncer at the door 🚪. It checks: *Are we still loading?* (show a spinning wheel). *Are you logged in?* If not, send you to the sign-in page. If yes, let you through (`<Outlet />` = "go ahead inside").
- **Lines 91–130** `App`: Here's the actual map:
- **Line 93** `<BrowserRouter>`: Turn on the "address bar understands pages" machine.
- **Line 98**: `/` → `LandingPage` (the welcome home page).
- **Lines 101–107**: `/sign-in`, `/request-access`, `/forgot-password`, `/reset-password`, `/verify-email` → all share one common frame (`AuthLayout` = same header/background). These are the "before you log in" doors. - **Lines 110–118**: `/repository`, `/title-similarity`, `/trend-analysis`, `/analytics`, `/profile`, `/settings` → all behind the **bouncer** (`ProtectedRoute`). You must be logged in. - **Line 121**: Any unknown address → the 404 `NotFound` page. - **Line 125**: One always-available "Upload Thesis" pop-up window, reachable from anywhere.

frontend/src/api/client.js — "The waiter who talks to the kitchen" 🍷🍷🍷

The frontend can't do secret work (like checking a password) itself — it asks the backend. This file is the **telephone** between them.

- **Line 17**: `import axios from 'axios';` — Get the messenger library that sends requests to a server.
- **Lines 19–25**: Create one trusted messenger (`client`) with rules: `baseUrl` (the kitchen's address), `withCredentials: true` ("always bring the secret cookie so the kitchen knows who you are"), `Content-Type: application/json` ("speak in JSON", the universal data language).
- **Line 28**: `let refreshing = null;` — A note saying "are we currently asking for a new key?" (starts as "no").
- **Lines 32–34**: Three empty note-spots: "how to get the current key," "what to do when we get a new one," "what to do if it fails." `AuthContext` will fill these in later.
- **Lines 40–44** `configureAuthCallbacks`: A setup function: `AuthContext` tells the messenger "here's how to grab the key, here's what to do on success/failure."
- **Lines 49–60 (Request interceptor)**: Before every message is sent, automatically staple your ID badge (`Bearer <token>`) onto it — *if* you have one. Like showing your membership card before ordering.
- **Lines 65–72 (Response interceptor, success)**: When a message comes back fine, ring a little bell (`axios: request: success`) so the app knows "user is active, don't log them out from boredom."
- **Lines 73–124 (Response interceptor, error)**: If the kitchen says `**401` = "your key

expired": mark this request "already retried" (so we don't loop forever), single-flight** (if a refresh is already happening, wait for it instead of asking twice), ask the kitchen for a fresh key (/auth/refresh/), staple the *new* key on and re-send your original order. If even the refresh fails: the helpers onRefreshFailure() clear your login, and we send you back to the sign-in page. Finally, erase the "refreshing" note so the next person can try.

- **Line 127:** export default client; — Hand this finished messenger to the rest of the app to use.

Part 2 — The login brain (frontend)

frontend/src/context/AuthContext.jsx — "The brain that remembers who you are" ■

Imagine a name-tag clip on the wall. Everyone in the restaurant checks it to know if you're a member.

- **Lines 1–9 (comment):** This is the *real* login memory. The secret key lives only in the computer's short-term memory (never written on a sticky note/localStorage). The "stay-logged-in" key is a special cookie the browser hides from JavaScript.
- **Line 11:** import { createContext, useContext, useState, useEffect, useCallback } from 'react';
— Get five React tools: a shared memory box (createContext), a way to read it (useContext), sticky notes that change (useState), "do this after something" (useEffect), and "remember this recipe" (useCallback).
- **Line 12:** import client, { configureAuthCallbacks } from '../api/client'; — Bring the waiter (the client from the previous file).
- **Line 13:** import { clearAllCaches } from '../utils/appCaches'; — Bring a "erase the chalkboard" tool so a new person doesn't see the old person's notes.
- **Line 15:** const AuthContext = createContext(undefined); — Make an empty shared box called AuthContext. (undefined) = "nothing inside yet."
- **Line 17:** export function AuthProvider({ children }) { — Here's the Provider — the thing that wraps the app (we saw it in main.jsx, the sweater layer). children = everything inside it.
- **Lines 18–20:** Three sticky notes: user = who you are (starts empty), accessToken = your current entry key (starts empty), isInitializing = "are we still figuring out if you're logged in?" (starts true = yes, still checking).
- **Line 22:** const isAuthenticated = !!accessToken && !!user; — You count as "logged in" only if you have BOTH a key AND a known user. !! just turns it into a yes/no answer.
- **Lines 28–43 loadMe:** Go ask the kitchen "who am I?" ■. It calls GET /auth/me/.
If a token is given, staple it on. The kitchen replies with your profile; we save it in user. If it fails, we forget you (set both to empty) and admit the failure.
- **Lines 48–59 (useEffect):** As soon as this provider turns on, tell the waiter (the client): "Here's how to grab the current key, and here's what to do when you get a new one or fail." This connects the waiter to this brain.
- **Lines 64–87 (silent refresh on startup):** When the app first opens, *quietly* try to

log you back in using the hidden cookie: call `POST /auth/refresh/`, save the new key, load your profile with that key, if anything fails (bad cookie, kitchen on fire, no internet) → forget you, **finally** flip `isInitializing` to `false` so the bouncer stops showing the spinner. (The comment explains a past bug: they used to only forget you on certain errors, which wrongly kicked people out when the wifi blipped. Now they forget you on **any** failure — safer.)

- **Lines 92–109** `signIn`: When you type email + password + maybe "remember me": send them to `POST /auth/login/`, grab the key from the reply, save it, load your profile, if it fails, forget everything and pass the error up so the page can show "wrong password."
- **Lines 114–127** `signOut`: Tell the kitchen `POST /auth/logout/` (best effort — even if that fails we still log you out locally). **Finally** erase all cached pages (so the next person can't peek at your data) and clear key + user.
- **Lines 132–143** `refresh`: Manually ask for a fresh key. Same idea as silent refresh but called on purpose.
- **Lines 145–154**: Pack everything into one `value` box: the user, the key, the yes/no flags, and all four actions (`signIn`, `signOut`, `refresh`, `loadMe`).
- **Lines 156–160**: Hand that box to every children inside the provider.
- **Lines 163–169** `useAuthContext`: A safe way to **open** the box. If someone tries to use it outside the provider (no sweater on), shout an error: "You forgot to wrap me in `AuthProvider!`"

frontend/src/hooks/useAuth.js — "A nickname" ■■

- **Line 10**: `export { useAuthContext as useAuth } from '../context/AuthContext';` — Let people type `useAuth` instead of the long `useAuthContext`. It's just an alias — same thing, shorter name. (Like calling refrigerator just fridge.)

Part 3 — The login kitchen (backend)

backend/auth_service/serializers.py — "The form checker at the door" ■

- **Lines 1–7 (comment)**: Before the kitchen does any real work, this file just checks the **shape** of what you sent. It makes sure your email looks like a school email. It does NOT check your password yet — that happens later, on purpose, to avoid a sneaky trick.
- **Line 9**: `from __future__ import annotations` — A small setting so the file can use modern type hints.
- **Line 11**: `from rest_framework import serializers` — Get Django REST's form-checking tools.
- **Lines 13–16**: `from common.validators import (...)` — Get the rule: "is this a school email?" and the error name for a wrong-domain email.
- **Lines 19–24** `LoginSerializer`: Describe the login form: `email` (must look like an email, max 254 letters), `password` (text, hidden when echoed back, can't be blank, max 512), `remember_me` (a yes/no, optional, defaults to no).

- **Lines 26–33** `validate_email`: When checking the email: trim spaces and make it lowercase, ask "is this a school email (ends with `pampangastateu.edu.ph`)?" If not → shout `INVALID_EMAIL_DOMAIN`, otherwise accept the cleaned version.

Why check email here but password later? The comment explains: to avoid **enumeration leak** — a trick where a hacker tells if an email exists by the **type** of error. By checking the password in the view the same way every time, you can't tell which part was wrong.

backend/auth_service/views.py — "The login counter in the kitchen" ■■■■

- **Lines 1–7 (comment)**: These are the actual login/refresh/logout/me actions. The hard token work is delegated to helper files. Every cookie-using action also checks the request came from our own website (anti-fake-origin guard).
- **Lines 9–35**: A pile of `import` lines — Collect all the tools we'll use: settings, time, response helpers, the User model, password checkers, the audit logger (a security diary), the origin guard, error types, rate limiters, and cookie helpers.
- **Lines 42–54** `_reject_unknown_fields`: If someone sends a field we didn't ask for (like sneaking in `is_admin: true`), reject it with `"UNKNOWN_FIELD"`. Keeps the form strict.
- **Lines 57–64** `_user_payload`: Build the small "here's who you are" card: id, email, first/last name, role.
- **Lines 71–73 (decorators on LoginView)**: Three guards taped above the login door:
 1. `require_origin_match` — only accept requests from our own site,
 2. `rate_limit_per_ip(10, 60, ...)` — one IP can try only **10 times per 60 seconds**,
 3. `rate_limit_per_email_on_failure(5, 15*60, ...)` — after **5 wrong tries on an email**, lock it for **15 minutes**. (These stop hackers from guessing passwords thousands of times a second.)
- **Lines 74–83** `LoginView`: The login counter. `AllowAny` = anyone may **try**. No built-in auth needed yet.
- **Lines 85–88**: First, reject unknown extra fields.
- **Lines 90–106**: Check the form is valid (email format, etc.). If the email field is bad, return a tidy error; otherwise a generic "invalid" error. We keep errors vague on purpose.
- **Lines 108–110**: Pull out the cleaned email, password, and `remember_me`.
- **Lines 112–115**: Look up the user by email. If found and active, check the password (`accounts_verify_password`). `needs_rehash` = "should we upgrade how we stored this password?"
- **Lines 117–133**: If the password is wrong: write in the security diary `auth.login.failure` (note the email tried and the IP — for safety investigators), then reply "Email or password is incorrect." with code `INVALID_CREDENTIALS`. Same message whether the email didn't exist or the password was wrong — no hints!
- **Lines 135–136**: If the password-checker says "please re-store this password more safely" (`needs_rehash`), do it quietly.
- **Lines 138–139**: Stamp `last_login_at` = right now, and save just those two fields.
- **Line 141**: Ask the token service to **issue a token pair** (a short key + a long hidden cookie key).

- **Lines 143–150:** Build the reply: the access key, "Bearer" type, how many seconds it lasts, and your user card. Then **bake the refresh key into a hidden cookie** on the reply (`set_refresh_cookie`).
 - **Lines 152–159:** Write `auth.login.success` in the diary (with your id and the "family id" of this session). Then send the reply.
 - **Lines 167–173** `RefreshView`: The "give me a new short key" counter. Same origin guard; allows **60 refreshes per 60 seconds per IP**.
 - **Lines 175–182:** Read the hidden refresh cookie. If it's missing → say "REVOKED / cookie missing".
 - **Lines 184–206:** Try to rotate the refresh key. Three possible problems: **Expired** → "your long key timed out," **Reuse detected** → "someone tried to use an old key twice — we threw away the WHOLE session family" (this is the security feature that forced `main.jsx` to drop `StrictMode!`), **Revoked** → "this key was cancelled." Each writes a diary entry.
 - **Lines 208–215:** If all good: reply with a new access key and bake a fresh refresh cookie.
 - **Lines 222–240** `LogoutView`: Logout: read the cookie, if present cancel that one key row (`revoke_one`), reply "204 no content", and **wipe the cookie** (`clear_refresh_cookie`). If a key was actually cancelled, note `auth.logout` in the diary. Friendly: even if the cookie was already gone, logout still succeeds.
 - **Lines 247–258** `MeView`: The "who am I?" counter. `IsAuthenticated` = you must already be logged in. It replies with your user card **plus** your list of permissions (what you're allowed to do, based on your role).
-

Part 4 — Inside the login kitchen (backend `auth_service` internals)

backend/auth_service/models.py — "The key notebook" ■

This is the **notebook** where the kitchen records every "stay-logged-in" key. The most important safety rule: only a **scrambled** version of the key is written down — never the key itself.

- **Lines 1–11 (comment):** The notebook stores "opaque refresh tokens." Each key has a "family." When you log in you get one key; when it's replaced (rotated) the new key joins the same family. If someone re-uses an **old** key, the whole family is cancelled.
- **Lines 13–18:** Imports — `uuid` (random IDs), Django settings, and the model tools.
- **Lines 21–28** `class RefreshToken`: The table. `REVOKED_REASON_CHOICES` is the list of reasons a key was cancelled: `rotated`, `logout`, `reuse_detected`, `password_reset`, `admin_revoke`.
- **Line 30:** `id` — every row gets a random UUID name tag.
- **Lines 31–36:** `user` — which person this key belongs to. If that person is deleted, their keys are deleted too (CASCADE).
- **Line 37:** `token_hash` — the **scrambled fingerprint** of the key (64 letters,

unique). The real key is never saved — only this fingerprint.

- **Line 38:** `family_id` — the "family group" ID. All keys from one login share it.
- **Lines 39–46:** `parent` — points at the *previous* key, like a family tree. When we replace a key, the new one's parent is the old one.
- **Line 47:** `remember_me` — did the user tick "keep me logged in?"
- **Line 48:** `expires_at` — the date/time the key stops working.
- **Lines 49–55:** `revoked_at` (when it was cancelled, empty if still good) and `revoked_reason` (one of the listed reasons, or empty).
- **Lines 56–58:** `created_at`, `user_agent` (which browser), `ip_address` (which computer) — written down for the security diary.
- **Lines 60–88** `class Meta`: Extra rules for the table: its real name is `refresh_tokens`; it builds **indexes** (fast lookup lists) for "this user's live keys" and "this family"; and a **check constraint** that `revoked_reason` must be one of the five reasons or empty (no typos allowed in the diary).
- **Lines 90–91:** `__str__` — how to print a row as text (used in logs).

backend/auth_service/services.py — "The token factory" ■

This file is the machine that **makes and cancels** keys.

- **Lines 1–11 (comment):** Lists the five jobs this factory does: issue a pair (at login), rotate (at refresh), revoke a whole family, revoke everything for a user, revoke just one key (logout).
- **Lines 13–31:** Imports and grabbing helper tools (the JWT maker, the opaque-token maker, the scrambler sha256).
- **Lines 34–38:** The five cancel-reason labels, written as constants so we don't mistype them.
- **Lines 41–47** `IssuedTokenPair`: A small labeled box that holds: the access key, the plain refresh key (the real one, only given to the browser), and the notebook row we created.
- **Lines 50–55** `_ttl_for`: "How long should the refresh key live?" Longer if `remember_me` is on.
- **Lines 58–71** `_client_ip / _user_agent`: Figure out *which computer* and *which browser* is asking, for the diary. (The X-Forwarded-For trick just reads the first IP if the request came through a proxy.)
- **Lines 74–90** `issue_token_pair`: At login: make an access JWT, make a random refresh key, store only its **fingerprint** in a fresh notebook row (new family ID, no parent), and hand back the pair.
- **Lines 93–97** `revoke_family`: Cancel **every still-good key** in a family in one database update. (Used when we suspect a stolen key.)
- **Lines 100–104** `revoke_all_for_user`: Cancel every key for one person (used by password reset — when you change your password, all old sessions die).

- **Lines 107–175** `rotate_refresh`: The clever one. It exchanges an old key for a new pair, and watches for theft: - **Line 121**: Scramble the presented key into a fingerprint. - **Line 126**: Open a **locked drawer** (`transaction.atomic`) so two requests can't trip over each other. - **Lines 128–130**: Find the notebook row by fingerprint; if not found → "Revoked." - **Line 132–133**: If expired → "Expired." - **Lines 135–144**: If already cancelled: **only** if the reason was rotated do we treat it as **theft (reuse)** — remember the family to kill it; otherwise just "Revoked." - **Lines 145–169**: If active → **rotate**: mark the old row cancelled (reason rotated), make a brand-new key row that **inherits the same family_id and remember_me**, issue a new access key, and return the pair. - **Lines 174–175**: After leaving the locked drawer, if we detected reuse, cancel the **whole family** and shout `RefreshReuseDetected`. (This is the rule that forced `main.jsx` to drop `StrictMode` — a test double-run looked like theft and booted everyone!)
- **Lines 178–192** `revoke_one`: Logout — cancel just this one row. Friendly: it does **not** error if the key is missing/expired/already cancelled.

backend/auth_service/authentication.py — "The badge checker" ■

- **Lines 1–14 (comment)**: This reads the `Authorization: Bearer <key>` header and checks the key is real. If the key is **expired**, it raises a special error the frontend recognizes so it can quietly fetch a new one. (Imports are done lazily inside the function to avoid a "circular import" tangle — two files each needing the other.)
- **Lines 18–25**: Imports and `class JWTAuthentication` with `keyword = 'Bearer'`.
- **Lines 27–60** `authenticate`:
- **Line 29–35**: Lazy imports (`User`, the `expired-error`, the `JWT verifier`).
- **Line 37**: Read the `Authorization` header.
- **Line 38–39**: If there's no header → return `None` ("not logged in yet; the bouncer will decide"). - **Lines 41–47**: Split the header; it must say `Bearer`, and have exactly the key (not zero, not three parts) — otherwise complain. - **Line 49–55**: Take the raw key, verify it. If expired → raise the special expired error (frontend refreshes). If invalid → fail. - **Lines 57–59**: Look up the user by the ID hidden in the key; if missing or turned off → fail. - **Line 60**: Return (`user`, `payload`) — "yes, this person is who they claim to be."
- **Lines 62–63** `authenticate_header`: Just returns the word `Bearer` for error messages.

backend/auth_service/urls.py — "The door signs" ■

- **Lines 1–6 (comment)**: This file maps web addresses to the counters. The school-login (SSO) doors live in their own spot so swapping in a real school provider later won't disturb the normal login.
- **Lines 8–11**: Import the `path` tool and the `counters/views` (including the SSO views).
- **Lines 14–21** `urlpatterns`: The list of doors:
- `login/` → `LoginView`
- `logout/` → `LogoutView`
- `refresh/` → `RefreshView`
- `me/` → `MeView`
- `sso/initiate/` and `sso/callback/` → the school-login views

Document continues — more files are appended as the explanation grows.